



C++ - Módulo 02

Polimorfismo ad-hoc, sobrecarga de operadores y la
forma de clase canónica ortodoxa

Resumen: Este documento contiene los ejercicios del Módulo 02 de los módulos de C++.

Versión: 9.1

Contenido

I	Introducción	2
II	Reglas generales	3
III	Nuevas reglas	6
Instrucciones de IA	IV	7
V	Ejercicio 00: Mi primera clase en forma canónica ortodoxa	9
VI	Ejercicio 01: Hacia una clase de números de punto fijo más útil	11
VII	Ejercicio 02: Ahora sí que estamos hablando	13
VIII	Ejercicio 03: BSP	15
IX	Presentación y evaluación por pares	17

Capítulo I

Introducción

C++ es un lenguaje de programación de propósito general creado por Bjarne Stroustrup como una extensión del lenguaje de programación C, o "C con clases" (fuente: [Wikipedia](#)).

El objetivo de estos módulos es introducirte en la Programación Orientada a Objetos. Este será el punto de partida de tu experiencia con C++. Muchos lenguajes se recomiendan para aprender programación orientada a objetos, pero decidimos elegir C++ porque deriva de tu viejo amigo C. Dado que es un lenguaje complejo, y para simplificar las cosas, tu código cumplirá con el estándar C++98.

Sabemos que el C++ moderno es muy diferente en muchos aspectos. Por lo tanto, si quieres convertirte en un desarrollador competente de C++, ¡de ti depende ir más allá de los 42 estándares Common Core!

Capítulo II

Reglas generales

Compilando

- Compila tu código con `c++` y los indicadores `-Wall -Wextra -Werror`
- Su código aún debería compilarse si agrega el indicador `-std=c++98`

Convenciones de formato y nomenclatura

- Los directorios de ejercicios se nombrarán de esta manera: `ex00`, `ex01`, ... , `exn`
- Nombra tus archivos, clases, funciones, funciones miembro y atributos según sea necesario en las directrices.
- Escriba los nombres de clase en formato Mayúsculas y minúsculas . Los archivos que contienen código de clase siempre se nombrarán según el nombre de la clase.
Por ejemplo: `ClassName.hpp/ClassName.h`, `ClassName.cpp` o `ClassName.hpp`. Si tiene un archivo de encabezado que contiene la definición de la clase "BrickWall", que representa un muro de ladrillos, su nombre será `BrickWall.hpp`.
- A menos que se especifique lo contrario, cada mensaje de salida debe terminar con un carácter de nueva línea y mostrarse en la salida estándar.

¡ Adiós, Norminette! No se impone ningún estilo de programación en los módulos de C++. Puedes seguir tu favorito. Pero recuerda que el código que tus pares evaluadores no pueden entender es código que no pueden calificar. Haz todo lo posible por escribir código limpio y legible.

Permitido/Prohibido

Ya no estás programando en C. ¡Hora de usar C++! Por lo tanto:

- Puedes usar casi todo lo de la biblioteca estándar. Por lo tanto, en lugar de limitarte a lo que ya sabes, sería inteligente usar las versiones C++ de las funciones de C que conoces siempre que sea posible.

Sin embargo, no puedes usar ninguna otra biblioteca externa. Esto significa que C++11 (y sus derivados) y las bibliotecas de Boost están prohibidas. Las siguientes funciones también están prohibidas: `*printf()`, `*alloc()` y `free()`. Si las usas, tu calificación será 0 y punto.

- Tenga en cuenta que, a menos que se indique explícitamente lo contrario, el uso del espacio de nombres `<ns_name>` y Las palabras clave de amistad están prohibidas. De lo contrario, tu calificación será -42.
- Solo se permite usar el STL en los módulos 08 y 09. Esto significa: no se permiten contenedores (vectoriales, de lista, de mapa, etc.) ni algoritmos (cualquier cosa que requiera incluir el encabezado `<algorithm>`) hasta entonces. De lo contrario, su calificación será de -42.

Algunos requisitos de diseño

- La pérdida de memoria también ocurre en C++. Cuando se asigna memoria (mediante el nuevo palabra clave), debe evitar fugas de memoria.
- Desde el Módulo 02 hasta el Módulo 09, sus clases deben estar diseñadas en el modelo Ortodoxo. Forma canónica, salvo que se indique explícitamente lo contrario.
- Cualquier implementación de función colocada en un archivo de encabezado (excepto las plantillas de función) significa 0 para el ejercicio.

Debes poder usar cada encabezado independientemente. Por lo tanto, deben incluir todas las dependencias necesarias. Sin embargo, debes evitar el problema de la doble inclusión añadiendo protecciones de inclusión. De lo contrario, tu calificación será 0.

Léame

- Puedes agregar archivos adicionales si lo necesitas (por ejemplo, para dividir tu código). Dado que estas tareas no están verificadas por un programa, puedes hacerlo siempre y cuando entregues los archivos obligatorios.
- A veces, las pautas de un ejercicio parecen cortas pero los ejemplos pueden mostrar requisitos que no están escritos explícitamente en las instrucciones.
- ¡Lee cada módulo completo antes de empezar! ¡De verdad, hazlo!

¡Por Odín, por Thor! ¡Usa tu cerebro!



Respecto al Makefile para proyectos C++, se aplican las mismas reglas que en C (ver el capítulo de Normas sobre el Makefile).



Tendrás que implementar muchas clases. Esto puede parecer tedioso, a menos que puedas programar tu editor de texto favorito.



Se le da una cierta cantidad de libertad para completar los ejercicios.

Sin embargo, sigue las reglas obligatorias y no seas perezoso.

¡Te pierdes mucha información útil! No dudes en leer sobre...

conceptos teóricos.

Capítulo III

Nuevas reglas

A partir de ahora, todas sus clases deben diseñarse en la forma canónica ortodoxa, a menos que se indique explícitamente lo contrario. Implementarán las cuatro funciones miembro requeridas a continuación:

- Constructor predeterminado
- Constructor de copias
- Operador de asignación de copia
- Destructor

Divide el código de tu clase en dos archivos. El archivo de encabezado (.hpp/.h) contiene la clase. definición, mientras que el archivo fuente (.cpp) contiene la implementación.

Capítulo IV

Instrucciones de IA

- Context

Este proyecto está diseñado para ayudarle a descubrir los componentes fundamentales de su entrenamiento 42.

Para anclar adecuadamente los conocimientos y habilidades clave, es esencial adoptar un enfoque reflexivo al utilizar las herramientas y el soporte de IA.

El verdadero aprendizaje fundamental requiere un esfuerzo intelectual genuino, a través del desafío, la repetición y los intercambios de aprendizaje entre pares.

Para obtener una descripción más completa de nuestra postura sobre la IA (como herramienta de aprendizaje, como parte de la capacitación 42 y como una expectativa en el mercado laboral), consulte las preguntas frecuentes específicas en la intranet.

- Mensaje principal

Construye bases sólidas sin atajos.

Desarrollar realmente habilidades técnicas y de potencia.

Experimente el verdadero aprendizaje entre pares, comience a aprender a aprender y a resolver nuevos problemas.

El recorrido de aprendizaje es más importante que el resultado.

Conozca los riesgos asociados con la IA y desarrolle prácticas de control efectivas y contramedidas para evitar errores comunes.

- Reglas del alumno:

- Debes aplicar el razonamiento a las tareas asignadas, especialmente antes de recurrir a la IA.

- No debes pedir respuestas directas a la IA.
- Debes conocer los 42 enfoques globales sobre IA.

• Resultados de la fase:

Dentro de esta fase fundamental, obtendrás los siguientes resultados:

- Obtenga bases técnicas y de codificación adecuadas.
- Conozca por qué y cómo la IA puede ser peligrosa durante esta fase.

• Comentarios y ejemplo:

Sí, sabemos que la IA existe, y sí, puede resolver tus proyectos. Pero estás aquí para aprender, no para demostrar que la IA ha aprendido. No pierdas tu tiempo (ni el nuestro) solo para demostrar que la IA puede resolver el problema.

Aprender a los 42 años no se trata de saber la respuesta, sino de desarrollar la capacidad de encontrarla. La IA te da la respuesta directamente, pero eso te impide desarrollar tu propio razonamiento. Y razonar requiere tiempo, esfuerzo e implica fracaso. El camino al éxito no se supone que sea fácil.

- Tenga en cuenta que durante los exámenes, la IA no está disponible (no hay Internet, ni teléfonos inteligentes, etc.). Rápidamente se dará cuenta si ha confiado demasiado en la IA en su proceso de aprendizaje.

El aprendizaje entre pares te expone a diferentes ideas y enfoques, mejorando tus habilidades interpersonales y tu capacidad de pensar de forma divergente. Esto es mucho más valioso que simplemente chatear con un bot. Así que no seas tímido: ¡habla, pregunta y aprende con otros!

Sí, la IA formará parte del plan de estudios, tanto como herramienta de aprendizaje como tema en sí mismo. Incluso tendrás la oportunidad de crear tu propio software de IA. Para obtener más información sobre nuestro enfoque progresivo, consulta la documentación disponible en la intranet.

Buenas prácticas:

Estoy atascado con un nuevo concepto. Le pregunto a alguien cercano cómo lo abordó. Hablamos durante 10 minutos y, de repente, todo encaja. Lo entiendo.

Mala práctica:

Uso IA en secreto y copio código que parece correcto. Durante la evaluación por pares, no puedo explicar nada. Suspenso. Durante el examen, sin IA, me quedo atascado de nuevo. Suspenso.

Capítulo V

Ejercicio 00: Mi primera clase en Forma canónica ortodoxa

	Ejercicio: 00
Mi primera clase en la forma canónica ortodoxa	
Directorio: ex00/	
Archivos a enviar: Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp Prohibido: Ninguno	

Crees que sabes de números enteros y de punto flotante. ¡Qué monada!

Por favor lea este artículo de 3 páginas ([1](#), [2](#), [3](#)) para descubrir que no. Adelante, léelo.

Hasta hoy, cada número que usabas en tu código era básicamente un número entero o de punto flotante, o cualquiera de sus variantes (short, char, long, double, etc.). Después de leer el artículo anterior, es seguro asumir que los números enteros y los números de punto flotante tienen características opuestas.

Pero hoy, las cosas cambiarán. Descubrirás un nuevo y asombroso tipo de número: ¡ los números de punto fijo! Siempre ausentes en los tipos escalares de la mayoría de los lenguajes, los números de punto fijo ofrecen un valioso equilibrio entre rendimiento, exactitud, alcance y precisión. Esto explica por qué son particularmente aplicables a los gráficos por computadora, el procesamiento de sonido o la programación científica, por nombrar solo algunos.

Como C++ no tiene números de punto fijo, los agregarás. [Este artículo](#) Desde Berkeley es un buen comienzo. Si no tienes idea de qué es la Universidad de Berkeley, lee [esta sección](#). de su página de Wikipedia.

Cree una clase en forma canónica ortodoxa que represente un número de punto fijo:

- Miembros privados:
 - Un entero para almacenar el valor del número de punto fijo.
 - Una constante entera estática para almacenar la cantidad de bits fraccionarios. Su valor siempre será el literal entero 8.
- Miembros públicos:
 - Un constructor predeterminado que inicializa el valor del número de punto fijo en 0.
 - Un constructor de copias.
 - Una sobrecarga del operador de asignación de copia.
 - Un destructor.
 - Una función miembro `int getRawBits( void ) const;` que devuelve el valor sin procesar del valor de punto fijo.
 - Una función miembro `void setRawBits( int const raw );` que establece el valor sin procesar del número de punto fijo.

Ejecutando este código:

```
#include <iostream>

entero principal( void ) {

    Arreglado a;
    b( a ) fijo;
    C fijo;

    c = b;

    std::cout << a.getRawBits() << std::endl; std::cout << b.getRawBits() <<
    std::endl; std::cout << c.getRawBits() << std::endl;

    devuelve 0;
}
```

Debería generar algo similar a lo siguiente:

```
$> ./a.fuera
Constructor predeterminado llamado
Constructor de copia llamado
Operador de asignación de copia llamado // <-- Esta línea puede faltar dependiendo de su implementación de la función miembro getRawBits llamada

Constructor predeterminado llamado
Operador de asignación de copia llamado getRawBits
función miembro llamada getRawBits función miembro
llamada 0

Función miembro getRawBits llamada 0

Función miembro getRawBits llamada 0

Destructor llamado
Destructor llamado
Destructor llamado
$>
```

Capítulo VI

Ejercicio 01: Hacia una clase de números de punto fijo más útil

	Ejercicio 01
Hacia una clase de números de punto fijo más útil	
Directorio: ex01/	
Archivos a enviar: Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp Autorizado: roundf (de <cmath>)	

El ejercicio anterior fue un buen comienzo, pero nuestra clase es bastante inútil. Solo puede representar el valor 0.0.

Agregue los siguientes constructores públicos y funciones miembro públicas a su clase:

- Un constructor que toma un entero constante como parámetro.
Lo convierte al valor de punto fijo correspondiente. El valor de los bits fraccionarios debe inicializarse en 8, como en el ejercicio 00.
- Un constructor que toma un número de punto flotante constante como parámetro.
Lo convierte al valor de punto fijo correspondiente. El valor de los bits fraccionarios debe inicializarse en 8, como en el ejercicio 00.
- Una función miembro `float toFloat( void ) const;`
que convierte el valor de punto fijo en un valor de punto flotante.
- Una función miembro `int toInt( void ) const;`
que convierte el valor de punto fijo en un valor entero.

Y agregue la siguiente función a los archivos de clase `Fixed` :

- Una sobrecarga del operador de inserción (`<<`) que inserta una representación de punto flotante del número de punto fijo en el objeto de flujo de salida pasado como parámetro.

C++ - Módulo 02

Ejecutando este código:

```
#include <iostream>

entero principal( void ) {

    Fijado a;

    Constante fija b( 10 );
    Constante fija c( 42.42f );
    Constante fija d( b );

    a = Fijo( 1234.4321f );

    std::cout << "a es std::cout " << a << std::endl;
    << "b es " << b << std::endl;
    std::cout << "c es std::cout " << c << std::endl;
    << "d es " << d << std::endl;

    std::cout << "a es " << a.toInt() << " como entero" << std::endl;
    std::cout << "b es std::cout " << b.toInt() << " como entero" << std::endl;
    << "c es std::cout << "d es " << d.toInt() << " como entero" << std::endl;
    << " como entero" << std::endl;

    devuelve 0;
}
```

Debería generar algo similar a lo siguiente:

```
$> ./a.fuera
Constructor predeterminado llamado
Constructor int llamado
Constructor flotante llamado
Constructor de copia llamado
Operador de asignación de copia llamado
Constructor flotante llamado
Operador de asignación de copia llamado
Destructor llamado
a es 1234,43
b es 10
c es 42.4219
d es 10
a es 1234 como entero
b es 10 como entero
c es 42 como entero
d es 10 como entero
Destructor llamado
Destructor llamado
Destructor llamado
Destructor llamado
Destructor llamado
$>
```

Capítulo VII

Ejercicio 02: Ahora estamos hablando

	Ejercicio02
Ahora estamos hablando	
Directorio: ex02/	
Archivos a enviar: Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp Autorizado: roundf (de <cmath>)	

Agregue funciones miembro públicas a su clase para sobrecargar los siguientes operadores:

- Los 6 operadores de comparación: $>$, $<$, $>=$, $<=$, $==$ y $!=$.
- Los 4 operadores aritméticos: $+$, $-$, $*$ y $/$.
- Los 4 operadores de incremento/decremento (pre-incremento y post-incremento, pre-decremento y post-decremento), que aumentarán o disminuirán el valor de punto fijo en el más pequeño representable, tal que $1 + > 1$.

Agregue estas cuatro funciones miembro públicas sobrecargadas a su clase:

- Una función miembro estática `min` que toma dos referencias a números de punto fijo como parámetros y devuelve una referencia al más pequeño.
- Una función miembro estática `min` que toma dos referencias a un punto fijo constante números como parámetros y devuelve una referencia al más pequeño.
- Una función miembro estática `max` que toma dos referencias a números de punto fijo como parámetros y devuelve una referencia al mayor.
- Una función miembro estática `max` que toma dos referencias a un punto fijo constante números como parámetros y devuelve una referencia al mayor.

Depende de usted probar cada característica de su clase. Sin embargo, al ejecutar el código a continuación:

```
#include <iostream>

entero principal( void ) {

    Fijado          a;
    Constante fija   b( Fijo( 5.05f ) * Fijo( 2 ) );

    std::cout << a << std::endl; std::cout << ++a <<
    std::endl; std::cout << a << std::endl; std::cout << a++
    << std::endl; std::cout << a << std::endl;

    std::cout << b << std::endl;

    std::cout << Fijo::max( a, b ) << std::endl;

    devuelve 0;
}
```

Debería generar algo como (para mayor legibilidad, el mensaje del constructor/destructor)
Los sabios se eliminan en el siguiente ejemplo):

```
$> ./a.out 0

0.00390625
0.00390625
0.00390625
0,0078125
10,1016
10.1016 $>
```



Si alguna vez realiza una división por 0, es aceptable que el programa
accidentes

Capítulo VIII

Ejercicio 03: BSP

	Ejercicio 03
BSP	
Directorio: ex03/	
Archivos a enviar: Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp, Point.{h, hpp}, Point.cpp, bsp.cpp Autorizado: roundf (de <cmath>)	

Ahora que tienes una clase Fixed funcional , sería bueno usarla.

Implementa una función que indique si un punto está dentro de un triángulo o no. Muy útil, ¿verdad?



BSP significa Partición de Espacio Binario. De nada. :)



Puedes aprobar este módulo sin completar el ejercicio 03.

Comencemos creando la clase Punto en forma canónica ortodoxa que representa un punto 2D:

- Miembros privados:

- Un atributo constante fijo x.
- Un atributo constante fijo y.
- Cualquier otra cosa útil.

- Miembros públicos:

- Un constructor predeterminado que inicializa x e y en 0.
- Un constructor que toma dos números de punto flotante constantes como parámetros. Inicializa x e y con esos parámetros.
- Un constructor de copias.
- Una sobrecarga del operador de asignación de copia.
- Un destructor.
- Cualquier otra cosa útil.

Para concluir, implemente la siguiente función en el archivo apropiado:

```
bool bsp( Punto constante a, Punto constante b, Punto constante c, Punto constante punto);
```

- a, b, c: Los vértices de nuestro querido triángulo.
- punto: El punto a comprobar.
- Devuelve: Verdadero si el punto está dentro del triángulo. Falso en caso contrario.
Por lo tanto, si el punto es un vértice o está en una arista, devolverá Falso.

Implemente y entregue sus propias pruebas para asegurarse de que su clase se comporte como se espera.

Capítulo IX

Presentación y evaluación por pares

Entregue su tarea en su repositorio de Git como de costumbre. Solo se evaluará el trabajo dentro de su repositorio durante la defensa. No dude en verificar los nombres de sus carpetas y archivos para asegurarse de que sean correctos.

Durante la evaluación, ocasionalmente se podría solicitar una breve modificación del proyecto. Esto podría implicar un cambio menor de comportamiento, la escritura o reescritura de algunas líneas de código, o una función fácil de añadir.

Si bien este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar su comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que elija (por ejemplo, su configuración habitual) y debería ser factible en unos pocos minutos, a menos que se defina un período de tiempo específico como parte de la evaluación.

Por ejemplo, se le puede pedir que realice una pequeña actualización a una función o script, modifique una pantalla o ajuste una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.